

NTU SPACE DATA CENTER

NTU SPACE DYNAMICS

Software Module Design Report

Orbit • Attitude • Solar Environment • Power and Thermal • Access Analysis

Institute	College of Computing and Data Science, Nanyang Technological University
Author	Qin Jiahao
Reviewer	Liu Xiyuan
Issue date	17 August 2026

College of Computing and Data Science, Nanyang Technological University

Version Control

Abstract	This report defines the architecture, algorithms, public interfaces, data contracts, verification approach, and deployment constraints of the NTU Space Dynamics Python module.			
Keywords	digital twin; orbit dynamics; attitude dynamics; Sun environment; access analysis; verification			
Change Record				
Revision	Change Request	Date	Author	Description
1.0	Initial issue	2026-08-17	Qin Jiahao	Orbit dynamics module test report based on the retained outline.

TABLE OF CONTENTS

1 Document Overview	1
1.1 Purpose.....	1
1.2 Scope.....	1
1.3 Intended readers	1
1.4 Normative terminology	1
2 Requirements and Design Objectives	2
3 Overall Design	3
3.1 Layered architecture.....	3
3.2 Package decomposition.....	3
3.3 Shared data flow	4
4 Concise Orbital-Dynamics Background	4
4.1 Reference Frames.....	4
4.2 Reference Frames Transformation.....	6
4.3 Orbit propagation	8
4.4 Classical orbital elements.....	8
4.5 Non-spherical gravity.....	9
4.6 Sun's celestial position	9
4.7 Solar Radiation Pressure	10
4.8 Power and Thermal	11
4.9 Access Calculation Using Cubic Hermite Interpolation	12
5 Detailed Functional Design.....	15
5.1 Common data models	15
5.2 Orbit propagation	15
5.2.1 Two-body propagator.....	15
5.2.2 J2 propagator.....	16
5.2.3 SGP4 propagator.....	16
5.2.4 High-precision propagator	16
5.3 High-order Earth gravity	16
5.4 Element and Cartesian-state conversion	17

5.5 Coordinate and frame transformations.....	17
5.6 Attitude conversion and dynamics	18
5.7 Solar environment and radiation pressure.....	18
5.8 Power considering panel temperature	18
5.9 Access propagation and interpolation.....	19
6 Public Interface Design.....	19
6.1 General contracts	19
6.2 Primary calling interfaces	20
6.3 Minimal end-to-end call.....	20
7 End-to-End Operational Workflow.....	21
8 Numerical Design and Performance	23
9 Verification and Validation.....	23
10 Deployment and Configuration.....	24
10.1 Runtime.....	24
10.2 Installation and VS Code interpreter.....	24
11 Limitations and Planned Evolution.....	24
12 Requirements Traceability Matrix	26

1 Document Overview

1.1 Purpose

The purpose of this report is to provide an implementable and reviewable design baseline for the current NTU Space Dynamics software module. It explains what the module computes, how the major algorithms are composed, which frames and units are accepted, and what each public interface returns.

1.2 Scope

- Orbit-state and classical-element conversion, analytical and numerical orbit propagation, and TLE-based SGP4 propagation.
- Static high-order Earth gravity from bundled EGM2008 or external fully normalized ICGEM/GRACE coefficient files.
- GCRS, ITRS, and TEME state transformation, plus WGS-84 geodetic/ECEF conversion.
- Euler-angle conversion, quaternion rigid-body dynamics, Sun position and beta-angle analysis.
- Conical eclipse, solar irradiance, cannonball solar-radiation pressure, and one-node solar-panel thermal power.
- Ground-target access from direct orbit propagation or Hermite interpolation of sampled states.
- End-to-end computation and diagnostic plotting of position, velocity, beta angle, illumination, power, temperature, quaternion, and body-rate histories.

1.3 Intended readers

The report is written for spacecraft simulation developers, mission-analysis engineers, software integrators, verification personnel, and reviewers who need enough orbital-dynamics context to evaluate the design without requiring a full astrodynamics textbook treatment.

1.4 Normative terminology

Table 3 Terminology and frame conventions

Term	Meaning in this module
GCRS	Geocentric Celestial Reference System; the numerical-propagation inertial-frame contract.
ITRS / ECEF	Earth-fixed frame used for gravity synthesis and ground geometry.
TEME	True Equator, Mean Equinox frame returned by the SGP4 implementation.
Osculating elements	Instantaneous Keplerian elements derived from a Cartesian state.
Mean elements	Averaged orbital parameters represented by a TLE and interpreted by SGP4.

Term	Meaning in this module
HPOP	Composable high-precision Cowell propagation architecture; not a single undeclared fidelity level.
Access	Time interval satisfying Earth-clear line of sight, minimum elevation, sensor FOV, and optional range.

2 Requirements and Design Objectives

The software was designed around explicit model semantics and auditability. A calculation result must identify its time system, units, frame, force-model selection, gravity data, and numerical tolerances. The module therefore favors a small set of composable Python interfaces over an opaque monolithic simulation service.

Table 4 Principal design objectives

ID	Objective	Design response
DO-01	Complete requested functional coverage	Public APIs cover all requested orbit, attitude, solar, power, pressure, gravity, and access functions.
DO-02	Safe data contracts	SI units, aware UTC datetimes, and explicit GCRS/ITRS/TEME labels are enforced at public boundaries.
DO-03	Fidelity is selectable	Two-body, J2, SGP4, and HPOP represent distinct model semantics instead of a single hidden propagator.
DO-04	Reproducibility	EGM2008 provenance and SHA-256 are documented; IERS, ephemeris, truncation, and tolerance settings are task inputs.
DO-05	Offline operation	Astropy built-in ephemeris and locally available IERS tables support deterministic disconnected runs.
DO-06	Extensibility	HPOP accepts custom force models and gravity coefficients; torque can be constant or callable.
DO-07	Verification	Twenty-three unit tests cover invariants, reference vectors, transforms, gravity synthesis, solar geometry, thermal trends, and access events.

Design rule: A frame label is part of the numerical value. A TEME vector must never be renamed GCRS without a real transformation, and a velocity transform must include transport terms caused by Earth rotation.

3 Overall Design

3.1 Layered architecture

The module is organized as a layered scientific library. User applications and batch workflows call analysis services. Those services reuse common types, frame transformations, gravity synthesis, force models, integration, interpolation, and event-search utilities. External scientific libraries are isolated behind explicit frame and model contracts.

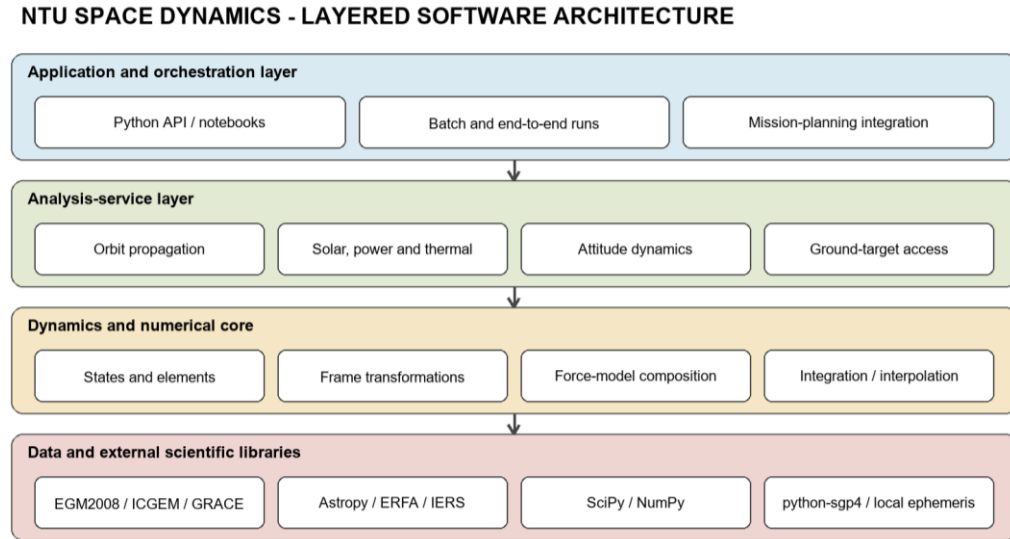


Figure 1 Layered software architecture

3.2 Package decomposition

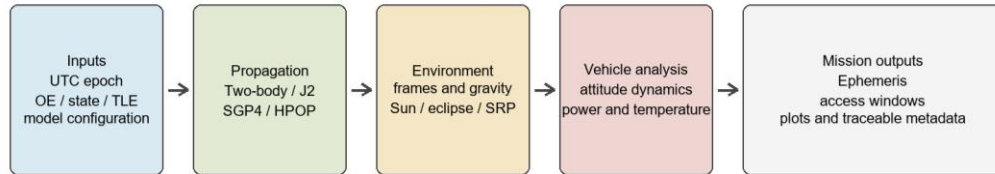
Table 5 Package-level decomposition

Package or module	Responsibility	Principal public API
types.py	Validated immutable data contracts	OrbitState, Ephemeris, ClassicalElements, AttitudeEphemeris
orbits/	Element conversion and propagation	oe_to_rv, rv_to_oe, TwoBodyPropagator, J2Propagator, SGP4Propagator, HighPrecisionPropagator
gravity.py	ICGEM loading, Earth orientation, spherical harmonics	GravityFieldCoefficients, EarthOrientationCache, SphericalHarmonicGravity
coordinates.py	Celestial/terrestrial transformations	eci_to_ecef, ecef_to_eci, transform_ephemeris, geodetic/ECEF conversion
attitude.py	Euler conversion and rigid-body propagation	ypr_to_euler, euler_to_dcm, EulerDynamics
solar.py	Sun, eclipse, beta angle, irradiance, SRP	sun_position_analytical, sun_position_ephemeris, eclipse_fraction, sun_intensity, sun_radiation_pressure
power.py	One-node panel thermal and electrical model	SolarPanel, power_considering_thermal

Package or module	Responsibility	Principal public API
access.py	Ground target/sensor geometry and event refinement	GroundTarget, NadirSensor, access_propagation, access_interpolation

3.3 Shared data flow

END-TO-END COMPUTATION FLOW



All public interfaces use SI units, timezone-aware UTC, and explicit GCRS / ITRS / TEME frame labels.

Figure 2 End-to-end computation flow

Orbit outputs are reused rather than re-derived by each subsystem. The same epoch sequence drives Sun vectors, eclipse, panel irradiance, attitude propagation, and access analysis. This alignment prevents hidden interpolation and frame mismatches between otherwise independent calculations.

4 Concise Orbital-Dynamics Background

4.1 Reference Frames

The orbit-dynamics module uses three reference frames to separate inertial orbit calculations, Earth-fixed geometry, and SGP4-compatible state representation. GCRS is used for numerical orbit states, ITRS for Earth-fixed gravity and targets, and TEME for raw SGP4 results.

The **Geocentric Celestial Reference System (GCRS)** is the primary frame for numerical orbit states. Its origin is located at the Earth's center of mass, while its axes remain kinematically non-rotating relative to the celestial reference system. Spacecraft position and velocity, numerical propagation results, and Sun and Moon direction vectors are represented in GCRS. Each state record contains an epoch, a three-dimensional position vector, a velocity vector, units, and an explicit frame identifier.

The **True Equator, Mean Equinox (TEME)** frame is retained for compatibility with the SGP4 propagator and Two-Line Element data. Its equatorial plane follows the true equator of date, while its reference direction is associated with the mean equinox of date. Raw SGP4 position and velocity outputs are expressed in TEME and remain in this frame until an

explicit conversion is performed. TEME states must therefore be transformed into GCRS for comparison with numerical orbit states or into ITRS for ground-access calculations.

The **International Terrestrial Reference System (ITRS)** is an Earth-fixed frame that rotates with the Earth. Its z -axis points toward the IERS Reference Pole, and its x -axis passes through the IERS Reference Meridian. The y -axis completes the right-handed coordinate system. Earth gravity coefficients, ground stations, surface targets, and geodetic longitude, latitude, and altitude are naturally defined in this frame. ITRS is therefore used to calculate ground access, elevation, azimuth, range, Earth-fixed illumination, and geographical ground tracks.

Table 1 Category and application of reference frames

Reference Frame	Category	Primary Applications
GCRS	Earth-centered inertial reference frame	Numerical orbit integration, Sun and Moon directions, and perturbation-force calculations
TEME	Earth-centered inertial-type reference frame based on a legacy approximation	Representation of raw SGP4 position and velocity outputs
ITRS	Earth-centered, Earth-fixed reference frame	Ground stations, ground targets, Earth gravity fields, subsatellite points, and access calculations

The GCRS-to-ITRS position transformation can be expressed as

$$\mathbf{r}_{\text{ITRS}} = \mathbf{W}(t)\mathbf{R}(t)\mathbf{Q}(t)\mathbf{r}_{\text{GCRS}},$$

where $\mathbf{Q}(t)$ accounts for precession and nutation, $\mathbf{R}(t)$ represents Earth rotation, and $\mathbf{W}(t)$ represents polar motion. This transformation requires Earth-orientation parameters, including UT1–UTC and polar-motion coordinates. The corresponding velocity transformation must also include the derivative of the rotation matrix

$$\mathbf{v}_{\text{ITRS}} = \mathbf{C}(t)\mathbf{v}_{\text{GCRS}} + \dot{\mathbf{C}}(t)\mathbf{r}_{\text{GCRS}},$$

where

$$\mathbf{C}(t) = \mathbf{W}(t)\mathbf{R}(t)\mathbf{Q}(t).$$

4.2 Reference Frames Transformation

A coordinate transformation changes the numerical components of a vector without changing the represented physical position or direction. For two Cartesian frames $\backslash(A\backslash)$ and $\backslash(B\backslash)$, the position transformation is

$$\mathbf{r}_B = \mathbf{C}_A^B(t) \mathbf{r}_A,$$

where $\mathbf{C}_A^B(t)$ is the direction cosine matrix from frame $\backslash(A\backslash)$ to frame $\backslash(B\backslash)$. For orthonormal reference frames,

$$(\mathbf{C}_A^B)^{-1} = (\mathbf{C}_A^B)^\top.$$

When the destination frame rotates relative to the source frame, the velocity transformation must include the time derivative of the rotation matrix:

$$\mathbf{v}_B = \mathbf{C}_A^B \mathbf{v}_A + \dot{\mathbf{C}}_A^B \mathbf{r}_A.$$

The second term is the transport term caused by frame rotation. Omitting this term produces incorrect Earth-fixed velocities, even when the transformed positions are correct.

A physical vector, such as gravitational acceleration evaluated at one instant, only requires a rotation of its components:

$$\mathbf{a}_B = \mathbf{C}_A^B \mathbf{a}_A.$$

Transport terms are required when transforming coordinate derivatives, rather than an independently evaluated physical vector.

GCRS and ITRS transformation: The Geocentric Celestial Reference System (GCRS) is used for numerical orbit states, while the International Terrestrial Reference System (ITRS) is used for Earth-fixed geometry. The transformation can be expressed as

$$\mathbf{r}_{\text{ITRS}} = \mathbf{W}(t) \mathbf{R}(t) \mathbf{Q}(t) \mathbf{r}_{\text{GCRS}},$$

where $\mathbf{Q}(t)$ represents the celestial-to-intermediate transformation, including precession, nutation, and celestial-pole motion, $\mathbf{R}(t)$ represents Earth rotation, evaluated using UT1, and $\mathbf{W}(t)$ represents polar motion between the terrestrial intermediate frame and ITRS. The complete transformation therefore depends on the observation epoch, the adopted precession-nutation model, UT1–UTC, polar-motion coordinates, and leap-second data.

TEME transformations: The True Equator, Mean Equinox (TEME) frame is used as the native output frame of SGP4. The project first parses the user-supplied TLE and propagates it with the Vallado SGP4 implementation:

$$\text{TLE} \xrightarrow{\text{SGP4}} (\mathbf{r}_{\text{TEME}}, \mathbf{v}_{\text{TEME}}).$$

TEME is a legacy, date-dependent reference frame associated with the true equator and mean equinox. Its axes are not identical to either GCRS or ITRS. Consequently, a TEME state cannot be converted by changing its frame label.

When an SGP4 result is compared with a numerical orbit, the required transformation is

$$\text{TEME} \rightarrow \text{GCRS}.$$

When an SGP4 result is used for ground-track or access calculations, the required transformation is

$$\text{TEME} \rightarrow \text{ITRS}.$$

WGS-84 geodetic and ITRS coordinates: Ground stations and ground targets are specified by geodetic latitude φ , longitude λ , and altitude h . These quantities are converted to ITRS/ECEF Cartesian coordinates using the WGS-84 reference ellipsoid. The first eccentricity squared and prime-vertical radius are

$$e^2 = f(2 - f)$$

$$N(\varphi) = \frac{a}{\sqrt{1 - e^2 \sin^2 \varphi}}$$

where a is the WGS-84 equatorial radius and f is the ellipsoid flattening. The Cartesian coordinates are

$$x = (N + h) \cos \varphi \cos \lambda,$$

$$y = (N + h) \cos \varphi \sin \lambda,$$

$$z = [N(1 - e^2) + h] \sin \varphi.$$

MOD-to-GCRS solar transformation: The analytical solar-position model initially produces the Earth-to-Sun vector in the mean equator and mean equinox of date frame (MOD). This vector must be converted before combination with GCRS spacecraft states:

$$\mathbf{r}_{\square}^{\text{MOD}} \rightarrow \mathbf{r}_{\square}^{\text{GCRS}}.$$

The project represents the analytical vector with Astropy's PrecessedGeocentric frame. Astropy then applies the required precession and frame transformation to produce a GCRS-compatible solar vector.

VVLH-to-GCRS transformation: The access module defines sensor directions in a local vertical, local horizontal frame using the project's VVLH convention. The local basis is constructed from the GCRS position and velocity:

$$\begin{aligned}\hat{\mathbf{e}}_z &= -\frac{\mathbf{r}}{\|\mathbf{r}\|}, \\ \hat{\mathbf{e}}_y &= \frac{\mathbf{r} \times \mathbf{v}}{\|\mathbf{r} \times \mathbf{v}\|}, \\ \hat{\mathbf{e}}_x &= \hat{\mathbf{e}}_y \times \hat{\mathbf{e}}_z.\end{aligned}$$

Here, $\hat{\mathbf{e}}_z$ points toward nadir and $\hat{\mathbf{e}}_y$ follows the orbit normal. A sensor boresight defined in VVLH is transformed into GCRS as

$$\mathbf{b}_{\text{GCRS}} = b_x \hat{\mathbf{e}}_x + b_y \hat{\mathbf{e}}_y + b_z \hat{\mathbf{e}}_z.$$

The resulting direction is compared with the GCRS target line of sight to calculate the sensor off-boresight angle.

4.3 Orbit propagation

An orbit is represented by position $\mathbf{r}$ and velocity $\mathbf{v}$ at an aware UTC epoch. Numerical propagation integrates the first-order state equations. The central term produces ideal Keplerian motion; each additional acceleration represents a declared perturbation.

$$\begin{cases} \frac{d\mathbf{r}}{dt} = \mathbf{v} \\ \frac{d\mathbf{v}}{dt} = -\mu \frac{\mathbf{r}}{r^3} + \sum a_p \end{cases}$$

Two-body propagation keeps only the central term. J2 adds the dominant oblateness effect. HPOP integrates a configurable sum of high-order gravity, third-body gravity, drag, solar pressure, relativity, and custom forces. SGP4 is different: it propagates the mean elements encoded in a TLE using the SGP4 analytical theory and returns TEME states.

4.4 Classical orbital elements

For elliptic motion the six classical elements are semi-major axis a , eccentricity e , inclination i , right ascension of the ascending node Ω , argument of periapsis ω , and true anomaly v . They provide a compact geometric description, but Ω and ω become singular for equatorial and circular orbits. The implementation uses deterministic conventions so that OE→RV→OE→RV reconstructs the original state.

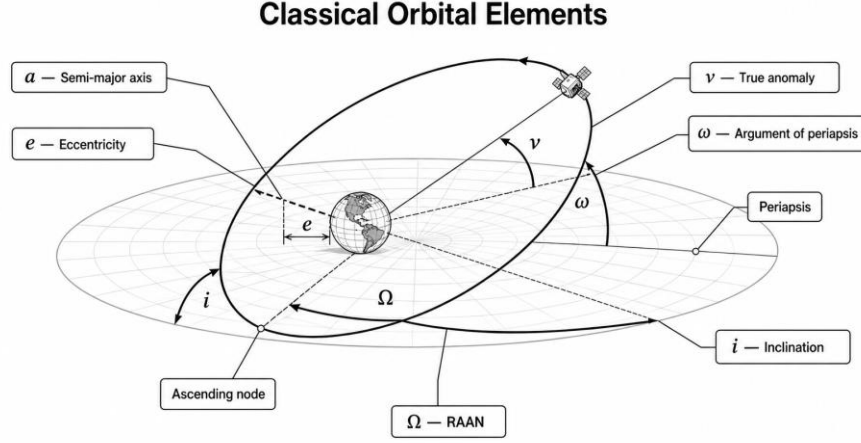


Figure 2 Demonstration of the classical orbital elements

4.5 Non-spherical gravity

Earth's mass distribution is represented by normalized spherical-harmonic coefficients $\bar{C}_{nm}$ and $\bar{S}_{nm}$. J_2 is the dominant degree-two zonal contribution. Higher degrees and orders represent progressively finer latitude/longitude structure. Gravity synthesis is performed in ITRS; the acceleration is then rotated back to GCRS for integration.

$$U(r, \varphi, \lambda) = \frac{\mu}{r} \sum_{n=0}^{\infty} \left(\frac{R}{r} \right)^n \sum_{m=0}^n \bar{P}_{nm}(\sin \varphi) [\bar{C}_{nm} \cos(m\lambda) + \bar{S}_{nm} \sin(m\lambda)].$$

The packaged EGM2008 subset is fully normalized and tide-free through degree/order 120. The HPOP default truncation is 70×70 . Increasing degree improves spatial detail but increases computation cost approximately with the number of retained coefficients.

4.6 Sun's celestial position

At each requested UTC epoch, the numerical Sun-position path converts time to TT/TDB and evaluates the barycentric positions of the Sun and Earth from the selected ephemeris: the offline Astropy solution or a configured JPL binary kernel. The apparent geocentric vector is formed with a light-time correction, so the solar state is evaluated at the retarded epoch.

$$\mathbf{r}_{\square/E}(t) = \mathbf{r}_{\square}(t_{\text{TDB}} - \tau) - \mathbf{r}_E(t_{\text{TDB}}), \quad \tau = \frac{\|\mathbf{r}_{\square/E}\|}{c}$$

The corrected vector is normalized in GCRS. Right ascension and declination are obtained from its Cartesian components; the same unit vector is rotated to ITRS with the precession-nutation, Earth-rotation, and polar-motion matrices when Earth-fixed illumination or access geometry is required.

$$\hat{\mathbf{s}}_{\text{GCRS}} = \frac{\mathbf{r}_{\square/E}}{\|\mathbf{r}_{\square/E}\|}, \quad \alpha = \text{atan2}(y, x), \quad \delta = \arcsin\left(\frac{z}{\|\mathbf{r}_{\square/E}\|}\right)$$

Model fidelity is controlled by the ephemeris source, leap-second data, and Earth-orientation parameters rather than by a propagation step size. The offline solution supports deterministic routine analyses; a version-controlled JPL kernel is selected when higher accuracy and reproducibility are required. Each result records the epoch, reference frame, ephemeris source, and kernel version.

4.7 Solar Radiation Pressure

Solar Radiation Pressure (SRP) is produced when solar photons transfer momentum to the spacecraft surface. The resulting force generally points away from the Sun. Its magnitude depends on the solar irradiance, spacecraft optical properties, effective illuminated area, spacecraft mass, Sun–spacecraft distance, and eclipse condition. Let $\mathbf{r}_\square$ and $\mathbf{r}_{\text{sc}}$ denote the positions of the Sun and spacecraft in the same inertial reference frame. The vector from the Sun to the spacecraft is $\mathbf{r}_{\square/\text{sc}} = \mathbf{r}_{\text{sc}} - \mathbf{r}_\square$, $d = \|\mathbf{r}_{\square/\text{sc}}\|$. The corresponding unit vector is

$$\mathbf{u}_{\text{SRP}} = \frac{\mathbf{r}_{\square/\text{sc}}}{\|\mathbf{r}_{\square/\text{sc}}\|}.$$

Because this vector points from the Sun toward the spacecraft, it also represents the direction of the SRP acceleration. Solar irradiance decreases according to the inverse-square law

$$S(d) = S_0 \left(\frac{\text{AU}}{d} \right)^2,$$

where $S_0 \approx 1361 \text{ W m}^{-2}$ is the solar irradiance at one astronomical unit. The corresponding radiation pressure is

$$P(d) = \frac{S(d)}{c} = P_0 \left(\frac{\text{AU}}{d} \right)^2,$$

where

$$P_0 = \frac{S_0}{c} \approx 4.56 \times 10^{-6} \text{ N m}^{-2}.$$

A common orbit-dynamics approximation is the cannonball model, in which the spacecraft is represented by a body with a constant reference area and a single radiation-pressure coefficient. The SRP force is

$$\mathbf{F}_{\text{SRP}} = \nu P_0 C_R A \left(\frac{\text{AU}}{d} \right)^2 \mathbf{u}_{\text{SRP}},$$

and the corresponding acceleration is

$$\mathbf{a}_{\text{SRP}} = \nu P_0 C_R \frac{A}{m} \left(\frac{\text{AU}}{d} \right)^2 \mathbf{u}_{\text{SRP}}$$

where ν is the illumination or visible-Sun fraction, C_R is the radiation-pressure coefficient, A is the reference illuminated area, m is the spacecraft mass, and A_m is the area-to-mass ratio. A perfectly absorbing surface has $C_R \approx 1$, while reflective surfaces have $C_R > 1$. The theoretical value approaches 2 for ideal specular reflection.

4.8 Power and Thermal

The incident solar power is calculated from the illuminated panel area, local solar irradiance, and solar incidence angle:

$$P_I = AI \max(0, \cos \theta).$$

The factor $\max(0, \cos \theta)$ prevents the rear side of the panel from generating power. The irradiance I should already include Sun-spacecraft distance scaling and the applicable eclipse factor. Therefore, P_I becomes zero when the panel is shadowed or faces away from the Sun. The electrical conversion efficiency varies with panel temperature according to a linear temperature model:

$$\eta(T) = \eta_{\text{ref}} [1 + k_T (T - T_{\text{ref}})].$$

The generated electrical power is then

$$P_E = \eta_{\text{ref}} [1 + k_T (T - T_{\text{ref}})] P_I.$$

Here, η_{ref} denotes the conversion efficiency at the reference temperature T_{ref} . The coefficient k_T is normally negative because photovoltaic efficiency decreases as the cell temperature increases. In the numerical implementation, the calculated efficiency should be constrained to the physically meaningful interval between zero and one. The panel temperature is calculated using a one-node lumped thermal model:

$$C \frac{dT}{dt} = \alpha P_{\text{incident}} - P_{\text{electric}} - \varepsilon \sigma A_{\text{rad}} (T^4 - T_{\text{space}}^4).$$

The first term on the right-hand side represents absorbed solar energy. The second term represents energy exported as electrical power, while the third term represents net radiative heat loss to deep space. Their difference defines the instantaneous net heating rate:

$$\dot{Q}_{\text{net}} = \alpha P_{\text{incident}} - P_{\text{electric}} - \varepsilon \sigma A_{\text{rad}} (T^4 - T_{\text{space}}^4).$$

The transient temperature is obtained by integrating

$$\frac{dT}{dt} = \frac{\dot{Q}_{\text{net}}}{C}$$

from the specified initial temperature. For example, a first-order time update can be expressed as

$$T_{n+1} = T_n + \frac{\Delta t}{C} \dot{Q}_{\text{net}}(T_n, t_n).$$

A higher-order ordinary differential equation integrator may be used when improved accuracy or numerical stability is required. The thermal capacity C controls the response rate, with a larger value producing a slower temperature response.

Under constant illumination and attitude conditions, the thermal equilibrium temperature is obtained by setting the temperature derivative to zero:

$$\alpha P_{\text{incident}} - \eta_{\text{ref}} \left[1 + k_T (T_{\text{eq}} - T_{\text{ref}}) \right] P_{\text{incident}} - \varepsilon \sigma A_{\text{rad}} (T_{\text{eq}}^4 - T_{\text{space}}^4) = 0.$$

Because this equation contains both a linear temperature-dependent efficiency term and a fourth-order radiation term, (T_{eq}) is normally determined using a numerical root solver. If the conversion efficiency is treated as constant, an explicit approximation is available:

$$T_{\text{eq}} = \left[T_{\text{space}}^4 + \frac{(\alpha - \eta) P_{\text{incident}}}{\varepsilon \sigma A_{\text{rad}}} \right]^{1/4}.$$

The transient temperature generally lags behind the instantaneous equilibrium temperature because of the panel's finite thermal capacity. The current single-node model neglects structural conduction, Earth infrared radiation, albedo, internal heat dissipation, battery losses, and maximum-power-point-tracking dynamics.

4.9 Access Calculation Using Cubic Hermite Interpolation

The orbit propagator provides the satellite position and velocity at discrete epochs. Consider two adjacent states, $(\mathbf{r}_k, \mathbf{v}_k)$ and $(\mathbf{r}_{k+1}, \mathbf{v}_{k+1})$, defined at t_k and t_{k+1} , respectively. The interval duration and normalized interpolation variable are defined as

$$\Delta t = t_{k+1} - t_k, \quad u = \frac{t - t_k}{\Delta t}, \quad 0 \leq u \leq 1.$$

Within this interval, the satellite position is represented by a cubic polynomial

$$\mathbf{r}(u) = \mathbf{a}_0 + \mathbf{a}_1 u + \mathbf{a}_2 u^2 + \mathbf{a}_3 u^3.$$

Differentiating the polynomial with respect to time gives the interpolated velocity:

$$\mathbf{v}(u) = \frac{d\mathbf{r}}{dt} = \frac{1}{\Delta t} (\mathbf{a}_1 + 2\mathbf{a}_2 u + 3\mathbf{a}_3 u^2).$$

The interpolant must reproduce both the position and velocity at the two endpoints:

$$\begin{aligned} \mathbf{r}(0) &= \mathbf{r}_k, & \mathbf{v}(0) &= \mathbf{v}_k, \\ \mathbf{r}(1) &= \mathbf{r}_{k+1}, & \mathbf{v}(1) &= \mathbf{v}_{k+1}. \end{aligned}$$

Substitution of these constraints into the cubic polynomial gives

$$\begin{aligned} \mathbf{a}_0 &= \mathbf{r}_k, & \mathbf{a}_1 &= \Delta t \mathbf{v}_k, \\ \mathbf{a}_0 + \mathbf{a}_1 + \mathbf{a}_2 + \mathbf{a}_3 &= \mathbf{r}_{k+1}, \\ \mathbf{a}_1 + 2\mathbf{a}_2 + 3\mathbf{a}_3 &= \Delta t \mathbf{v}_{k+1}. \end{aligned}$$

Solving these four endpoint equations gives the following remaining polynomial coefficients:

$$\begin{aligned} \mathbf{a}_2 &= 3(\mathbf{r}_{k+1} - \mathbf{r}_k) - \Delta t(2\mathbf{v}_k + \mathbf{v}_{k+1}), \\ \mathbf{a}_3 &= 2(\mathbf{r}_k - \mathbf{r}_{k+1}) + \Delta t(\mathbf{v}_k + \mathbf{v}_{k+1}). \end{aligned}$$

The resulting position interpolation can therefore be written in the standard cubic Hermite form:

$$\mathbf{r}(t) = H_{00}(u)\mathbf{r}_k + H_{10}(u)\Delta t \mathbf{v}_k + H_{01}(u)\mathbf{r}_{k+1} + H_{11}(u)\Delta t \mathbf{v}_{k+1},$$

where the Hermite basis functions are

$$\begin{aligned} H_{00}(u) &= 2u^3 - 3u^2 + 1, \\ H_{10}(u) &= u^3 - 2u^2 + u, \\ H_{01}(u) &= -2u^3 + 3u^2, \\ H_{11}(u) &= u^3 - u^2. \end{aligned}$$

The interpolated inertial position is subsequently transformed into the Earth-centered, Earth-fixed frame: $\mathbf{r}_s^{\text{ECEF}}(t) = \mathbf{C}_{\text{ECI}}^{\text{ECEF}}(t)\mathbf{r}(t)$, where $\mathbf{C}_{\text{ECI}}^{\text{ECEF}}(t)$ is the time-dependent transformation matrix. If the ground-station position is $\mathbf{r}_g^{\text{ECEF}}$, the station-to-satellite line-of-sight vector is

$$\boldsymbol{\rho}(t) = \mathbf{r}_s^{\text{ECEF}}(t) - \mathbf{r}_g^{\text{ECEF}}.$$

For a ground station at geodetic latitude φ_g and longitude λ_g , the line-of-sight vector is transformed into the local east-north-up frame:

$$\begin{bmatrix} \rho_E \\ \rho_N \\ \rho_U \end{bmatrix} = \begin{bmatrix} -\sin \lambda_g & \cos \lambda_g & 0 \\ -\sin \varphi_g \cos \lambda_g & -\sin \varphi_g \sin \lambda_g & \cos \varphi_g \\ \cos \varphi_g \cos \lambda_g & \cos \varphi_g \sin \lambda_g & \sin \varphi_g \end{bmatrix} \boldsymbol{\rho}(t).$$

Because ρ_U is the projection along the local zenith direction, the elevation angle satisfies

$$\sin E_l(t) = \frac{\rho_U}{\sqrt{\rho_E^2 + \rho_N^2 + \rho_U^2}}.$$

For improved numerical robustness near the horizon, the elevation is evaluated as

$$E_l(t) = \operatorname{atan} 2\left(\rho_U, \sqrt{\rho_E^2 + \rho_N^2}\right).$$

The visibility margin is defined as

$$g(t) = E_l(t) - E_{\min},$$

where $E_{\min}$ is the minimum permitted elevation angle. The binary visibility state is consequently

$$V(t) = \begin{cases} 1, & g(t) \geq 0, \\ 0, & g(t) < 0. \end{cases}$$

A visibility boundary exists in an interval $[t_i, t_{i+1}]$ when $g(t_i)g(t_{i+1}) < 0$. The boundary epoch t^* is obtained by solving $g(t^*) = 0$. A secant estimate of the boundary time can first be calculated from

$$t^{(0)} = t_i - g(t_i) \frac{t_{i+1} - t_i}{g(t_{i+1}) - g(t_i)}.$$

Brent's method then combines secant interpolation, inverse quadratic interpolation, and bisection to refine the root. The iteration terminates when $|t_b - t_a| < \varepsilon_t$, where ε_t is the required time tolerance. A negative-to-positive crossing of $g(t)$ represents Acquisition of Signal (AOS), whereas a positive-to-negative crossing represents Loss of Signal (LOS).

This method determines access boundaries without repeating the orbital propagation during root refinement. It can therefore achieve timing accuracy finer than the original ephemeris sampling interval at a relatively low computational cost. The ephemeris interval must remain sufficiently short to prevent an entire visibility window from occurring between two samples without a detectable sign change. Strictly, this implementation uses piecewise cubic Hermite interpolation because both position and velocity are constrained at every node. The resulting trajectory is continuously differentiable, although second-derivative continuity is not generally guaranteed.

5 Detailed Functional Design

5.1 Common data models

Table 6 Core data objects

Object	Stored data	Invariant or behavior
OrbitState	epoch, position_m[3], velocity_m_s[3], frame	Aware UTC; SI; frame is GCRS, ITRS, or TEME.
Ephemeris	times[N], positions_m[N,3], velocities_m_s[N,3], frame	Aligned lengths; monotonic time; state(i) reconstruction.
ClassicalElements	a, e, i, Ω , ω , v, optional epoch	Elliptic osculating elements; radians and metres.
AttitudeEphemeris	times_s, quaternion xyzw, body rates	Body-to-inertial active quaternion; rates in body frame.
ThermalPowerResult	temperature, electrical power, incident power	Arrays aligned to input elapsed times.
AccessWindow	start, end	Positive duration property.
AccessResult	windows plus sampled margins and geometry	Provides traceable event-search diagnostics.

5.2 Orbit propagation

Table 7 Propagator selection and model semantics

Propagator	Model	Frame	Primary use
TwoBodyPropagator	Exact elliptic Kepler solution from the initial osculating state	GCRS	Baseline, initialization, unit tests, short arcs
J2Propagator	Central gravity + J2, integrated by DOP853	GCRS	Dominant node/perigee drift and a light numerical baseline
SGP4Propagator	Vallado SGP4/SDP4 applied to TLE mean elements	TEME	Catalog/TLE propagation and operational screening
HighPrecisionPropagator	Cowell integration with composable force stack	GCRS	Engineering high-fidelity prediction and sensitivity studies

5.2.1 Two-body propagator

TwoBodyPropagator converts the initial Cartesian state to elliptic elements, advances the mean anomaly with the Keplerian mean motion, solves Kepler's equation, and converts the result back to Cartesian position and velocity. It is analytical and does not accumulate numerical integration error over a single ideal orbit, but it deliberately omits all perturbations.

5.2.2 J2 propagator

J2Propagator composes central gravity and the standard J2 acceleration and integrates the six-state Cartesian equations with SciPy DOP853. Users configure relative tolerance, position and velocity absolute tolerances, and maximum integration step. It excludes atmosphere, third bodies, SRP, and higher harmonics.

5.2.3 SGP4 propagator

SGP4Propagator accepts two TLE lines, validates the epoch and satellite record through python-sgp4, and returns SI-unit TME states. TLE age and element quality normally dominate the prediction error. The interface therefore keeps TME explicit and requires transform_ephemeris for GCRS or ITRS use.

5.2.4 High-precision propagator

HighPrecisionPropagator uses adaptive DOP853 Cowell integration. The default force stack is central gravity plus EGM2008 70×70, Sun/Moon third-body gravity, and Schwarzschild correction. When area-to-mass is positive, the stack also enables segmented exponential-atmosphere drag and cannonball SRP. The Sun and Moon ephemerides are shared across forces using sampled interpolation. Additional objects implementing acceleration(elapsed_s, state, epoch) can be appended through extra_forces.

Table 8 HPOP configuration

Parameter	Default	Design meaning
gravity_degree / order	70 / same as degree	Static harmonic truncation; None selects the J2 baseline path.
area_to_mass_m2_kg	0.0	Enables drag and SRP when positive.
drag_coefficient	2.2	Cannonball aerodynamic coefficient.
reflectivity_coefficient	1.3	Cannonball SRP coefficient.
include_third_body	True	Sun and Moon point-mass differential gravity.
include_relativity	True	Schwarzschild correction used by the engineering stack.
relative_tolerance	1e-11	DOP853 relative error control.
position / velocity tolerance	1 mm / 1 $\mu\text{m}\cdot\text{s}^{-1}$	Component-wise absolute error scaling.
max_step_s	120 s	Upper bound on adaptive integration step.

5.3 High-order Earth gravity

GravityFieldCoefficients stores fully normalized static cosine/sine coefficient matrices, model GM, reference radius, tide system, and provenance. from_icgem reads static gfc

records and supports explicit degree/order truncation. `load_builtin_egm2008` verifies and loads the packaged 0–120 subset.

`SphericalHarmonicGravity` computes the non-central potential and acceleration in the fixed frame. It intentionally excludes the degree-zero central term because HPOP adds central gravity separately using the model's GM. `EarthOrientationCache` precomputes GCRS→ITRS rotations and interpolates them with SLERP; the default node interval is 300 s.

Table 9 Bundled EGM2008 data contract

Attribute	Value
Packaged coverage	Degree and order 0–120
HPOP default	70×70
Normalization	Fully normalized
Tide system	Tide-free
Model GM	$3.986004415 \times 10^{14} \text{ m}^3/\text{s}^2$
Reference radius	6,378,136.3 m
Subset SHA-256	6ae505fcd7acfed84c5f69ff3e729be9cbb450162fd6bcf9116584c5b00570f2
External static models	ICGEM gfc records; GRACE monthly/static files after epoch reduction if required

Model limitation: The loader does not directly evaluate time-variable `gfct`, `trnd`, `asin`, or `acos` records. A GRACE monthly solution or other time-variable field must first be reduced to static coefficients for the task epoch.

5.4 Element and Cartesian-state conversion

`oe_to_rv` converts elliptic osculating classical elements to an inertial Cartesian state. `rv_to_oe` performs the inverse conversion and applies fixed conventions in singular cases: argument of periapsis is zero for circular inclined orbits; RAAN is zero for equatorial eccentric orbits; both are zero for circular equatorial orbits. The returned anomaly then represents argument of latitude or true longitude as appropriate.

5.5 Coordinate and frame transformations

`eci_to_ecef` converts a GCRS `OrbitState` to ITRS; `ecef_to_eci` performs the inverse. Both transport velocity, including the rotational contribution. `transform_ephemeris` extends the same contract to sampled GCRS, ITRS, or TEME ephemerides through Astropy/ERFA. WGS-84 `geodetic_to_ecef` and `ecef_to_geodetic` support target definition and ground geometry.

Operational data: Astropy transformations depend on UT1–UTC, polar motion, and leap seconds. Production runs should archive the controlled IERS Bulletin A/B table used for the calculation.

5.6 Attitude conversion and dynamics

`ypr_to_euler` interprets yaw-pitch-roll as aerospace intrinsic Z-Y-X rotation and converts it to a requested Euler sequence. Upper-case sequences are intrinsic and lower-case sequences are extrinsic, matching SciPy Rotation. `euler_to_dcm` returns the active direction-cosine matrix.

`EulerDynamics` integrates the body-to-inertial quaternion in SciPy xyzw order and the rigid-body Euler equation. Inertia may be supplied as principal moments or a matrix. Torque is expressed in the body frame and may be constant or a callable of time, quaternion, and body rate.

$$\dot{q} = 1/2 \cdot q \otimes [\omega_x, \omega_y, \omega_z, 0], \quad I\dot{\omega} = \tau - \omega \times (I\omega)$$

5.7 Solar environment and radiation pressure

Table 10 Solar functions

Function	Method	Output
<code>sun_position_analytical</code>	Low-order geocentric analytical Sun model; native MOD, optional GCRS conversion	Earth-to-Sun vector [m]
<code>sun_position_ephemeris</code>	Astropy built-in ephemeris or controlled local JPL kernel	GCRS-aligned Earth-to-Sun vector [m]
<code>sun_beta_angle</code>	Signed Sun elevation above the instantaneous orbit plane	rad or deg
<code>eclipse_fraction</code>	Apparent-disc overlap with conical umbra/penumbral geometry	Visible solar-disc fraction 0–1
<code>sun_intensity</code>	Solar constant scaled by actual Sun distance and optional eclipse fraction	Direct irradiance [W/m ²]
<code>sun_radiation_pressure</code>	Cannonball SRP, directed away from the Sun, optionally eclipsed	Acceleration [m/s ²]

The analytical and ephemeris Sun functions are separate because they serve different traceability and fidelity needs. The analytical model is small and deterministic. The ephemeris path can use Astropy's offline built-in solution or a task-controlled JPL binary kernel when `jplephem` is installed.

5.8 Power considering panel temperature

`SolarPanel` contains area, reference conversion efficiency, temperature coefficient, absorptivity, emissivity, radiating area, heat capacity, and deep-space sink temperature.

power_considering_thermal integrates a one-node lumped thermal balance. Incidence cosine is clipped to $[0,1]$; irradiance is expected to include eclipse before entering the model.

$$P_I = AI \max(0, \cos \theta),$$

$$P_E = \eta_{\text{ref}} (1 - k_T \|T - T_{\text{ref}}\|) P_I,$$

$$C \frac{dT}{dt} = \alpha P_I - P_E - \varepsilon \sigma A_{\text{rad}} (T^4 - T_{\text{space}}^4)$$

The panel is isothermal. Absorbed solar energy raises the node temperature after exported electrical power is removed; radiation to deep space cools the panel. Structural conduction, Earth infrared, albedo, battery behavior, and MPPT dynamics are outside the current model.

5.9 Access propagation and interpolation

GroundTarget defines a fixed WGS-84 latitude, longitude, altitude, and name. NadirSensor defines a fixed VVLH boresight through pitch and roll, a half-cone field of view, minimum target elevation, and optional maximum range. Access is the intersection of the target local-horizon margin, sensor off-boresight margin, range margin, and Earth-clear geometry.

access_propagation first samples any compatible orbit propagator and then searches for events. access_interpolation receives an existing GCRS Ephemeris and uses cubic Hermite interpolation with the stored velocity derivatives. Both scan at scan_step_s and refine each sign-changing boundary with Brent's method to boundary_tolerance_s.

Table 11 Access outputs

Output	Description
windows	Tuple of AccessWindow(start, end) objects with duration_s.
sample_times	Times used by the event scan.
combined_margin_rad	Minimum of all active geometric margins; positive means accessible.
elevation_rad	Target elevation in the local topocentric frame.
off_boresight_rad	Angle between sensor boresight and target line of sight.
slant_range_m	Spacecraft-to-target distance.

6 Public Interface Design

6.1 General contracts

- Lengths are metres, velocities are metres per second, elapsed times are seconds, torque is $\text{N}\cdot\text{m}$, temperature is kelvin, and power is watts.

- Angles are radians unless degrees=True or a from_degrees constructor is explicitly used.
- datetime values must be timezone-aware; UTC is the operational convention.
- Numerical propagation initial states must be GCRS; SGP4 outputs TEME; Earth-fixed calculations use ITRS.
- Functions validate dimensions, finite values, monotonic time arrays, physical parameter ranges, and compatible frames.

6.2 Primary calling interfaces

Table 12 Interface families

Family	Primary call	Input	Output
Orbit	propagator.propagate(initial, times) / propagate_grid(...)	OrbitState + epochs	Ephemeris
Elements	oe_to_rv(elements); rv_to_oe(r,v)	ClassicalElements or Cartesian state	Cartesian state or elements
Gravity	from_icgem(...); acceleration_fixed(r)	Coefficient file + ITRS position	Coefficients / acceleration
Frames	eci_to_ecef(state); transform_ephemeris(eph, target)	State or ephemeris	Transformed state/ephemeris
Attitude	EulerDynamics(...).propagate(q0, ω 0, times)	Inertia, torque, initial attitude	AttitudeEphemeris
Solar	Sun/beta/eclipse/intensity/SRP functions	Epoch, orbit state, area/mass	Vectors, angles, fractions, acceleration
Power	power_considering_thermal(times,I,cos θ ,panel)	Time histories + SolarPanel	ThermalPowerResult
Access	access_propagation(...)/ access_interpolation(...)	Orbit, target, sensor, event settings	AccessResult

6.3 Minimal end-to-end call

```

from datetime import datetime, timedelta, timezone
from ntu_space_dynamics import (
    ClassicalElements, HighPrecisionPropagator, OrbitState,
    SolarPanel, oe_to_rv, sun_position_ephemeris,
    eclipse_fraction, sun_intensity, power_considering_thermal,
)

epoch = datetime(2026, 1, 1, tzinfo=timezone.utc)
oe = ClassicalElements.from_degrees(
    7_000_000.0, 0.001, 97.5, 20.0, 0.0, 0.0, epoch
)
r0, v0 = oe_to_rv(oe)
initial = OrbitState(epoch, r0, v0, "GCRS")
eph = HighPrecisionPropagator(gravity_degree=70).propagate_grid(
    initial, epoch + timedelta(hours=3), step_s=60.0
)

```

The full example in examples/end_to_end.py adds Sun geometry, illumination, thermal power, rigid-body attitude, interpolated access, summary statistics, and multi-format plots.

7 End-to-End Operational Workflow

1. Create an aware UTC epoch and define the initial Cartesian state, classical elements, or TLE record.
2. Select the propagation semantics: two-body, J2, SGP4, or a declared HPOP force stack.
3. Generate one aligned ephemeris and transform frames only through the public coordinate interfaces.
4. Evaluate Sun vectors, beta angle, eclipse fraction, irradiance, SRP, thermal power, and attitude at the same epochs.
5. Evaluate access from the generated ephemeris, refining each open/close event boundary.
6. Export numerical results, model metadata, and diagnostic plots for engineering review.

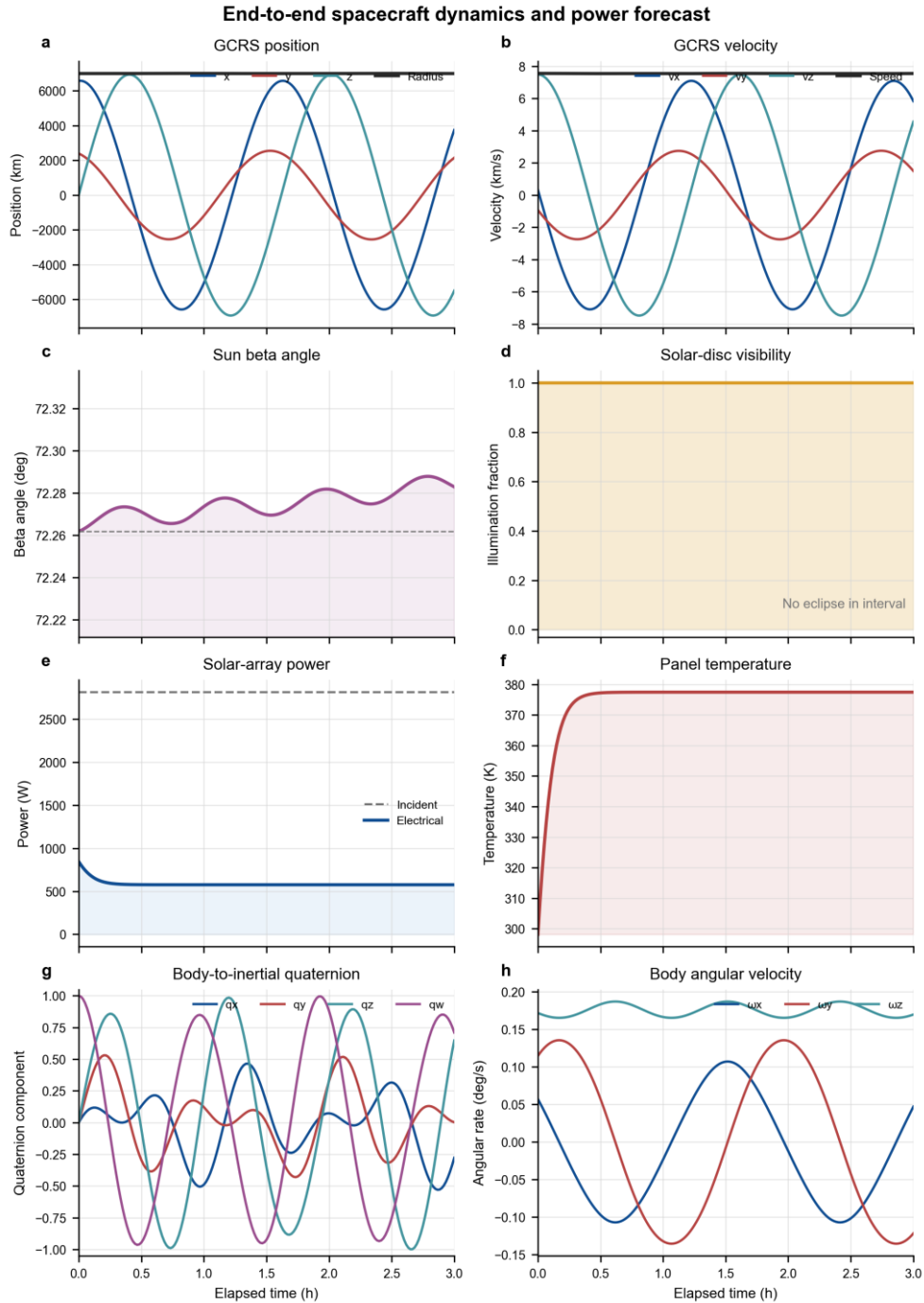


Figure 3 End-to-end propagation, solar, thermal-power, and attitude histories

The diagnostic figure is an engineering sanity-check surface rather than a formal orbit-determination validation product. It makes state discontinuities, eclipse transitions, unexpected power saturation, thermal divergence, quaternion normalization problems, and body-rate instability visible in one view.

8 Numerical Design and Performance

Table 13 Numerical methods and controls

Area	Method	Primary controls
Two-body	Analytical elliptic Kepler solution	Epoch grid and gravitational parameter
J2/HPOP	SciPy solve_ivp DOP853	rtol, position/velocity atol, max_step_s
Attitude/thermal	Adaptive non-stiff integration	Input sampling and tolerance configuration
Earth orientation	Astropy/ERFA rotation nodes + SLERP	IERS data and sample_step_s
Sun/Moon	Shared sampled ephemeris interpolation	Ephemeris source and sampling interval
Access state	Cubic Hermite interpolation	Orbit sample spacing and velocity quality
Access events	Fixed scan + Brent root refinement	scan_step_s and boundary_tolerance_s
Gravity	Normalized harmonic summation	Coefficient source, degree, order, tide system

High-order gravity is the main controllable cost in typical HPOP runs. Raising the degree/order increases both coefficient count and recurrence work. The recommended practice is to demonstrate convergence for the mission altitude and forecast duration, then select the lowest truncation that meets the error budget. Access scan_step_s must be smaller than the shortest window that must be detected.

9 Verification and Validation

The current baseline passes 23 unit tests under Python's unittest runner. Tests use analytical invariants, round trips, Vallado reference vectors, published gravity metadata, finite-difference checks, model degeneration, and event-boundary refinement. This is implementation verification; mission-specific validation still requires comparison with authoritative ephemerides, tracking data, or a trusted tool.

Table 14 Automated verification summary

Test group	Count	Representative evidence
Elements and propagators	7	OE/RV round trips including singular cases; two-body period closure; J2 energy; HPOP/J2 equivalence; Vallado SGP4 vector
Gravity	8	EGM metadata and zonals; $\bar{C}_{20} \rightarrow J_2$ reduction; finite-difference potential gradient; high-degree effect; ICGEM truncation; HPOP arc
Coordinates and solar	4	GCRS/ITRS position-velocity round trip; analytical/ephemeris Sun direction; eclipse; beta and SRP direction

Test group	Count	Representative evidence
Attitude, power, access	4	YPR rotation equivalence; torque-free invariants; thermal trend; refined access boundaries

Executed baseline: 23 tests passed on 17 August 2026. The end-to-end example also generated PNG, PDF, SVG, and TIFF diagnostic figures.

10 Deployment and Configuration

10.1 Runtime

- Python 3.10 or later.
- Required: NumPy ≥ 1.24 , SciPy ≥ 1.10 , Astropy ≥ 5.3 , Matplotlib ≥ 3.7 , python-sgp4 ≥ 2.23 .
- Optional: jplephem ≥ 2.18 for a controlled local JPL binary ephemeris kernel.
- Packaged data: EGM2008 tide-free static coefficients through degree/order 120 and SHA-256 sidecar.

10.2 Installation and VS Code interpreter

```
cd E:\MyProject\NTU_Space_Data_Center
python -m venv .venv
.\venv\Scripts\python.exe -m pip install -e .
.\venv\Scripts\python.exe -m unittest discover -s tests -v
```

In Visual Studio Code select Python: Select Interpreter and choose E:\MyProject\NTU_Space_Data_Center\.venv\Scripts\python.exe. Production deployments should archive the exact coefficient file, IERS table, local ephemeris kernel if used, package versions, and model configuration alongside each result.

11 Limitations and Planned Evolution

Table 15 Present limitations and recommended extensions

Subsystem	Current boundary	Recommended extension
Gravity	Static fully normalized field; no tides or in-run time-variable coefficients	Solid/ocean/pole tides; task-epoch GRACE evaluation; EOP/data version manager
Atmosphere	Segmented exponential density; no space weather response	NRLMSISE-00/JB2008 with archived F10.7, F10.7a, and Ap/Kp
SRP	Cannonball area/mass and single reflectivity coefficient	Faceted geometry, self-shadowing, Earth albedo and IR

NTU SPACE DATA CENTER

Subsystem	Current boundary	Recommended extension
Ephemerides	Astropy built-in by default	Controlled DE440/DE441 or SPICE kernel bundle
Thermal/power	One isothermal panel node; no conduction or electrical storage/control	Multi-node thermal network, Earth IR/albedo, battery, EPS, MPPT
Attitude	Rigid body with external torque; no actuator/sensor models	Reaction wheels, magnetorquers, disturbances, guidance and control loops
Access	Fixed target and fixed VVLH sensor; no terrain/refraction/cloud/maneuver time	Terrain horizon, atmosphere, keep-out constraints, slew and payload modes
Validation	Unit tests and engineering sanity plots	Cross-tool campaigns and mission-tracking residual analysis

12 Requirements Traceability Matrix

The matrix maps every requested capability to its implemented interface, computational model, primary output, and verification status.

Table 16 Functional requirements response

ID	Requested capability	Implemented interface	Design model	Primary output	Status
F-01	Twobody	TwoBodyPropagator	Elliptic Kepler analytical propagation	GCRS Ephemeris	Implemented / tested
F-02	J2	J2Propagator	Central gravity + J2, DOP853	GCRS Ephemeris	Implemented / tested
F-03	SGP4	SGP4Propagator	Vallado SGP4/SDP4, TLE mean elements	TEME Ephemeris	Implemented / Vallado vector tested
F-04	HPOP	HighPrecisionPropagator	Cowell + declared perturbation stack	GCRS Ephemeris	Implemented / baseline tested
F-05	High-order gravity	GravityFieldCoefficients; SphericalHarmonicGravity	EGM2008 70×70 default, packaged to 120; external static ICGEM/GRACE	Potential / acceleration / HPOP	Implemented / 8 gravity tests
F-06	OE2RV	oe_to_rv	Elliptic osculating elements to Cartesian state	r [m], v [m/s]	Implemented / round-trip tested
F-07	RV2OE	rv_to_oe	Cartesian state to elements with singular conventions	ClassicalElements	Implemented / singular cases tested
F-08	ECI2ECEF	eci_to_ecef; ecef_to_eci; transform_ephemeris	GCRS↔ITRS and TEME transforms with velocity	OrbitState / Ephemeris	Implemented / round-trip tested
F-09	YPR2EULER	ypr_to_euler; euler_to_dcm	Intrinsic Z-Y-X YPR and arbitrary Euler sequence	Euler angles / DCM	Implemented / rotation tested
F-10	EULERDYNAMICS	EulerDynamics	Quaternion kinematics + rigid-body Euler equation	AttitudeEphemeris	Implemented / invariants tested
F-11	SUN POSITION Analytical	sun_position_analytical	Low-order MOD model with optional GCRS conversion	Sun vector [m]	Implemented / direction tested
F-12	SUN POSITION ephemeris	sun_position_ephemeris	Astropy built-in or local JPL kernel	Sun vector [m]	Implemented / direction tested

ID	Requested capability	Implemented interface	Design model	Primary output	Status
F-13	SUN BETA ANGLE	sun_beta_angle	Signed Sun elevation above orbit plane	Angle	Implemented / sign tested
F-14	SUN INTENSITY	eclipse_fraction; sun_intensity	Conical eclipse and Sun-distance correction	0–1 fraction; W/m ²	Implemented / eclipse tested
F-15	Power considering thermal	SolarPanel; power_considering_thermal	One-node energy balance and temperature-dependent efficiency	ThermalPowerResult	Implemented / thermal trend tested
F-16	Sun radiation pressure	sun_radiation_pressure	Eclipsed cannonball SRP	Acceleration [m/s ²]	Implemented / direction tested
F-17	ACCESS PROPAGATION	access_propagation	Propagate then scan/refine target/sensor geometry	AccessResult	Implemented / event tested
F-18	ACCESS INTERPOLATION	access_interpolation	Cubic Hermite state interpolation + Brent refinement	AccessResult	Implemented / boundary tested
F-19	End-to-end diagnostics	examples/end_to_end.py	Aligned orbit, solar, power, thermal, attitude, access and plots	Statistics + PNG/PDF/SVG/TIFF	Implemented / executed

Appendix A — Public API Inventory

Table A-1 Exported names in `ntu_space_dynamics.__all__`

Public name	Kind	Contract summary
<code>AccessResult</code>	Type	Access windows and sampled geometry/margins.
<code>AccessWindow</code>	Type	Start/end UTC and derived duration.
<code>AttitudeEphemeris</code>	Type	Quaternion and body-rate time history.
<code>ClassicalElements</code>	Type	Elliptic osculating elements.
<code>Ephemeris</code>	Type	Sampled orbit positions and velocities.
<code>EulerDynamics</code>	Class	Rigid-body attitude propagation.
<code>EarthOrientationCache</code>	Class	Cached GCRS→ITRS rotations with SLERP.
<code>GroundTarget</code>	Type	Fixed WGS-84 ground point.
<code>GravityFieldCoefficients</code>	Type	Static fully normalized harmonic coefficients.
<code>HighPrecisionPropagator</code>	Class	Composable Cowell HPOP.
<code>J2Propagator</code>	Class	Central + J2 numerical propagation.
<code>NadirSensor</code>	Type	Fixed VVLH sensor and access constraints.
<code>OrbitState</code>	Type	Single-epoch SI Cartesian state.
<code>SGP4Propagator</code>	Class	TLE propagation to TEME.
<code>SolarPanel</code>	Type	Thermal/electrical panel parameters.
<code>SphericalHarmonicGravity</code>	Class	Static non-central gravity synthesis.
<code>ThermalPowerResult</code>	Type	Temperature and power histories.
<code>TwoBodyPropagator</code>	Class	Exact elliptic Kepler propagation.
<code>access_interpolation</code>	Function	Access from a sampled GCRS ephemeris.
<code>access_propagation</code>	Function	Propagate then compute access.
<code>ecef_to_eci</code>	Function	ITRS/ECEF to GCRS state.
<code>ecef_to_geodetic</code>	Function	WGS-84 ECEF to latitude/longitude/altitude.
<code>eci_to_ecef</code>	Function	GCRS to ITRS/ECEF state.
<code>eclipse_fraction</code>	Function	Visible solar-disc fraction.
<code>euler_to_dcm</code>	Function	Euler angles to active DCM.
<code>geodetic_to_ecef</code>	Function	WGS-84 latitude/longitude/altitude to ECEF.
<code>load_builtin_egm2008</code>	Function	Load packaged EGM2008 to degree 120.
<code>oe_to_rv</code>	Function	Elements to Cartesian state.

Public name	Kind	Contract summary
power_considering_thermal	Function	Panel temperature and electrical power.
rv_to_oe	Function	Cartesian state to classical elements.
sun_beta_angle	Function	Signed Sun/orbit-plane angle.
sun_intensity	Function	Direct irradiance with optional eclipse.
sun_position_analytical	Function	Analytical Sun vector.
sun_position_ephemeris	Function	Ephemeris Sun vector.
sun_radiation_pressure	Function	Cannonball SRP acceleration.
transform_ephemeris	Function	GCRS/ITRS/TEME sampled state transform.
ypr_to_euler	Function	Aerospace YPR to arbitrary Euler sequence.

Appendix B — Model-Selection Guidance

Table B-1 Recommended model selection

Question	Recommended starting point
Need a clean algorithm baseline or short initialization?	TwoBodyPropagator.
Need the dominant oblateness drift with a small force stack?	J2Propagator.
Starting from a TLE or catalog product?	SGP4Propagator; keep TEME explicit.
Need perturbation composition and high-order gravity?	HighPrecisionPropagator with declared coefficients, tolerances, and environmental inputs.
Need fast repeated access on an existing orbit?	access_interpolation with sufficiently dense samples.
Need access while selecting or comparing propagators?	access_propagation.
Need deterministic offline Sun calculations?	Astropy built-in ephemeris or the analytical model, depending on the error budget.
Need higher solar-system accuracy?	Local controlled JPL kernel with jplephem.

Appendix C — References

Pavlis, N. K. et al. (2012). The development and evaluation of the Earth Gravitational Model 2008 (EGM2008). *Journal of Geophysical Research*, 117, B04406.

Vallado, D. A. et al. (2006). Revisiting Spacetrack Report #3. AIAA/AAS Astrodynamics Specialist Conference.

IERS Conventions (2010), IERS Technical Note 36.

Montenbruck, O. and Gill, E. (2000). Satellite Orbits: Models, Methods and Applications.

Astropy, ERFA, SciPy, NumPy, Matplotlib, and python-sgp4 project documentation.

NTU Space Dynamics source documentation: README.md, docs/model-fidelity.md, docs/open-source-evaluation.md, and data/gravity/README.md.